INTERNAL DOCUMENT · CONFIDENTIAL

Librax 实验室自动化平台
软件详细设计说明书

Lab Research Analysis Exchange — 全流程实验室样本检测与分析系统

版本	状态	技术栈	编制日期
V1.0	评审中	Spring Boot 3.5 · Java 17 · MySQL · Redis	2025-11

修订历史

版本	日期	修改说明	作者
v0.1	2025-09	初始版本，整体架构定稿	yinan
v0.5	2025-10	补充设备管理、资源调度模块设计	yinan
v0.8	2025-11	单步调试、子流程、WebSocket 实时通信完善	yinan
V1.0	2025-11	全量设计说明书，正式发布	yinan

目录

1 项目概述与目标

2 系统架构设计

3 流程引擎模块详设

4 设备管理模块详设

5 实验室管理模块详设

6 资源调度模块详设

7 任务执行模块详设

8 系统基础模块详设

9 数据库设计

10 接口规范设计

11 WebSocket 通信设计

12 安全架构设计

13 性能与可靠性设计

14 部署架构设计

15 测试策略

1 项目概述与目标

1.1 系统定位

Librax 是一个面向实验室自动化场景的企业级综合管理平台，融合了 **流程编排引擎、设备通信层、样本全生命周期管理、资源调度** 和 **任务执行框架** 五大核心能力，旨在解决现代实验室中流程复杂、设备异构、数据割裂、人工依赖度高等核心痛点。

核心业务场景

- 样本从签收 → 处理 → 检测 → 归档全流程数字化
- 多类型仪器（pH 计、浊度仪、分析仪）统一接入与调度
- AGV 自动搬运与存储架位置精确管理
- 检测流程 DAG 编排，支持并行、条件分支、失败补偿
- 样本衍生追溯（原始/分裂/合并/取液）完整谱系
- 多区域资源共享配额与独占资源管理

设计目标

- 流程定义与执行分离，支持版本化管理
- 步骤组件可复用，支持单步独立调试
- 设备协议插件化（TCP/HTTP/串口/SDK/Mock）
- 资源独占与共享两种模式，分布式锁保证一致性
- 全链路可观测：执行日志、事件追踪、关键路径分析
- 多租户 SaaS 模式，数据隔离，权限精细控制

1.2 技术栈选型

☕ **Java** 17+ | 🌱 **Spring Boot** 3.5.5 | 🔒 **Spring Security** 6.5.2 | 🗄️ **MyBatis Plus** 3.5.12

🔴 **Redis / Redisson** 3.35.0 | 🐬 **MySQL** 5.7 / 8.0 | ⚡ **LiteFlow** 2.10.x | 📊 **JGraphT** DAG 引擎

🔣 **Aviator** 条件表达式 | ⏰ **Quartz** 2.5.0 | 🗺️ **MapStruct** 1.6.3 | ☕ **Caffeine** 本地缓存

🌐 **OkHttp** HTTP 设备客户端　📋 **EasyExcel** 导入导出　🔭 **SkyWalking** 9.5.0 APM

📘 **Springdoc** 2.8.9 OpenAPI

1.3 术语定义

术语	英文	说明
流水线	Pipeline	由若干步骤组成的 DAG 工作流定义，描述实验检测过程
流水线实例	PipelineExecution	一次流水线运行的完整生命周期记录
步骤定义	StepDefinition	可复用的原子操作模板（如 pH 测量、离心等）
步骤实例	StepExecution	某次执行中某步骤的单次尝试记录
执行上下文	ExecutionContext	流水线运行期间步骤间共享的变量空间
节点 ID	nodeId	流水线内步骤的唯一标识，用于 DAG 拓扑关系
区域	Zone	实验室物理区域划分，资源与权限管控的基本单元
补偿	Compensation	步骤失败后执行的回滚/撤销操作
任务	Task	可被设备/AGV/人/Bean 执行的工作单元
回调令牌	CallbackToken	用于验证外部系统回调合法性的随机字符串

2 系统架构设计

2.1 整体分层架构

接入层 · ACCESS LAYER

- REST API (Spring MVC)
- WebSocket (STOMP)
- 设备回调 Webhook
- Quartz 定时触发
- MQ 消息消费

业务层 · BUSINESS LAYER

- librax-module-flow（流程引擎）
- librax-module-device（设备管理）
- librax-module-lab（实验室管理）
- librax-module-resource（资源调度）
- librax-module-task（任务执行）
- librax-module-system（用户权限）

能力层 · CAPABILITY LAYER

- DAG 调度引擎 (JGraphT)
- 规则引擎 (Aviator)
- 流程编排 (LiteFlow)
- 分布式锁 (Redisson)
- 幂等保护 (Redis)
- 设备连接器工厂
- Job 调度 (Quartz)
- 消息队列抽象

框架层 · FRAMEWORK LAYER

- Security (认证/授权)
- Web (统一响应/异常)
- MyBatis (ORM/分页)
- Multi-Tenant (租户隔离)
- Data Permission (数据权限)
- Monitor (健康监控)
- Excel 导入导出

基础设施层 · INFRASTRUCTURE LAYER

- MySQL 5.7/8.0 (主从读写分离)
- Redis 5.0/6.0/7.0 (缓存+队列+锁)
- RocketMQ / RabbitMQ / Kafka (异步消息)
- MinIO / S3 (文件存储)
- SkyWalking (链路追踪)

2.2 Maven 多模块结构

```
librax/
├─ librax-dependencies/          # BOM 依赖管理（版本锁定）
├─ librax-framework/             # 横切框架能力
│   ├─ librax-common/            # 基础工具、常量、异常
│   ├─ librax-spring-boot-starter-web/       # Web 层封装
│   ├─ librax-spring-boot-starter-security/  # 安全认证授权
│   ├─ librax-spring-boot-starter-mybatis/   # MyBatis Plus 封装
│   ├─ librax-spring-boot-starter-redis/     # Redis/Redisson 客户端
│   ├─ librax-spring-boot-starter-job/       # Quartz 任务调度
│   ├─ librax-spring-boot-starter-mq/        # 消息队列抽象
│   ├─ librax-spring-boot-starter-websocket/ # WebSocket 通信
│   ├─ librax-spring-boot-starter-monitor/   # SkyWalking/SpringAdmin
│   ├─ librax-spring-boot-starter-protection/# 幂等/限流/分布式锁
│   └─ librax-spring-boot-starter-excel/     # Excel 导入导出
├─ librax-module-system/         # 用户/角色/权限/租户/字典
├─ librax-module-infra/          # 代码生成/文件服务/配置中心
├─ librax-module-flow/           # 流程引擎（核心模块）
├─ librax-module-flow-api/       # 流程模块对外 API 契约
├─ librax-module-device/         # 设备管理与通信
├─ librax-module-device-api/     # 设备模块对外 API 契约
├─ librax-module-lab/            # 样本/材料/容器管理
├─ librax-module-resource/       # 资源调度与存储架管理
├─ librax-module-task/           # 任务队列与执行框架
└─ librax-server/                # 聚合启动入口（Spring Boot 主应用）
```

2.3 模块依赖关系

> 各业务模块通过 `-api` 子模块暴露对外接口契约，避免直接依赖内部实现，降低模块耦合度。

模块	依赖模块	说明
`librax-server`	所有业务模块	聚合部署入口
`module-flow`	infra, flow-api, device-api, job, mq, websocket	流程引擎，依赖设备回调 API
`module-device`	infra, device-api, flow-api	设备管理，依赖流程回调 API
`module-lab`	infra, flow	样本管理，关联流程执行
`module-resource`	infra	资源调度，无复杂外部依赖
`module-task`	infra	任务模型，被 flow/device 调用

模块	依赖模块	说明
`module-system`	infra	基础用户权限体系
`module-infra`	system	代码生成、文件服务等基础设施

3 流程引擎模块 (librax-module-flow)

3.1 模块职责

流程引擎是 Librax 的核心，负责实验检测流程的 **定义、版本管理、调度执行、状态追踪、失败补偿** 全生命周期管理。基于 DAG（有向无环图）模型，支持步骤并行执行、条件分支、子流程嵌套和单步独立调试。

3.2 流水线执行状态机

PENDING → RUNNING → PAUSED ↔ RUNNING → SUCCESS

RUNNING → FAILED → COMPENSATING → COMPENSATED

RUNNING → CANCELLED

3.3 步骤类型设计

🔬

INSTRUMENT

调用物理仪器，创建 Task 并等待设备回调，异步完成

⚙️

COMPUTE

执行 Spring Bean 方法或 LiteFlow 规则链，同步返回结果

🔀

CONDITION

使用 Aviator 表达式求值，根据结果选择后续分支路径

⏳

WAIT

等待定时器超时或外部信号（DEVICE_CALLBACK / EXTERNAL_EVENT）

🔔

NOTIFY

发送通知（短信/邮件/Webhook），不阻塞后续步骤

🧬

SAMPLE_SPLIT

样本分裂，创建子样本并为每个子样本启动独立分支执行

👤

🚀

MANUAL

暂停等待人工审批，支持指定审批人和截止时间

UNIT_LAUNCHER

启动子流程（子 PipelineExecution），支持嵌套工作流

3.4 DAG 调度引擎设计

DagScheduler 核心逻辑

1. 从 `PipelineStepDO` 构建有向图（节点=stepKey，边=precedingNodeIds）
2. 使用 **JGraphT 拓扑排序**验证无环性，输出执行序列
3. 遍历就绪节点（前驱全部 SUCCESS/SKIPPED），并行提交至线程池
4. 每个节点由 `StepExecutorFactory` 按 `stepType` 选择对应执行器
5. 执行完成后更新 `StepExecutionDO` 状态，触发下游节点就绪检查
6. 所有节点完成后，用 JGraphT 计算 **关键路径**，写入 `PipelineExecutionDO.criticalPath`

3.5 执行上下文（Context）设计

流水线执行期间，步骤间数据通过 `ExecutionContextDO` 共享，遵循以下命名空间规则：

命名空间	表达式示例	来源
`input`	`${input.sampleType}`	流水线启动时传入的参数
`ctx`	`${ctx.batchNo}`	步骤写入的上下文变量
`output.{stepKey}`	`${output.ph_measure.value}`	某步骤的输出字段（由 outputFields 定义）

3.6 失败补偿策略

补偿策略类型

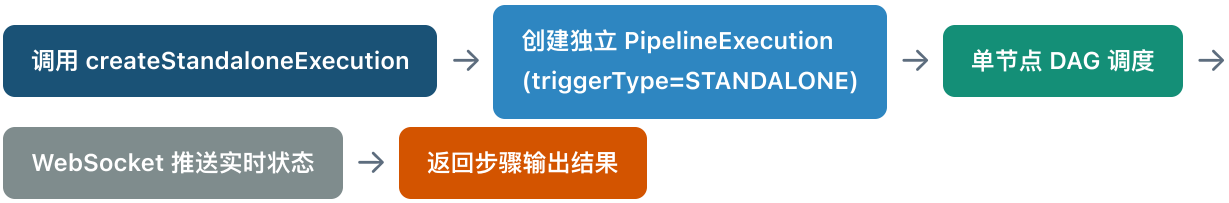
- **NONE**：失败后不执行补偿，直接标记失败
- **ON_FAIL**：仅在步骤失败时触发补偿步骤
- **ALWAYS**：无论成功失败，执行结束后均触发补偿

补偿执行机制

- 按逆拓扑顺序依次执行已完成步骤的 compensateStepKey
- 补偿步骤以 `runMode=COMPENSATE` 标识
- 创建新的 `StepExecutionDO` 记录补偿过程
- 补偿失败记录为 DEAD，人工干预

3.7 单步独立调试

系统支持对任意已标记 `runnableStandalone=true` 的步骤进行独立执行，用于开发验证和问题排查，无需启动完整流水线：



> Mock 模式：当设备未接入时，使用 `StepDefinitionDO.mockOutput` 模拟仪器返回数据，使调试不依赖物理设备。

3.8 核心数据模型

PipelineDefinitionDO — 流水线定义 `pd_pipeline_definition`

字段	类型	说明
pipelineKey	VARCHAR	业务唯一标识（如 `water_quality_test`），同 key 多版本
version	INT	版本号，同 key 递增，执行时冻结
failStrategy	ENUM	FAIL_FAST（一步失败整体停止）/ CONTINUE_ON_FAIL（继续其他并行步骤）
compensateStrategy	ENUM	NONE / ON_FAIL / ALWAYS
sampleMode	ENUM	REQUIRED（必须绑定样本）/ OPTIONAL / NONE

字段	类型	说明
sampleBindNodes	JSON	需要绑定样本的节点 ID 列表
status	ENUM	DRAFT / ACTIVE / DISABLED（只有 ACTIVE 可触发执行）

StepDefinitionDO — 步骤定义（可复用组件）

`pd_step_definition`

字段	类型	说明
stepKey	VARCHAR	步骤唯一标识（如 `ph_measure`），跨流水线复用
stepType	ENUM	INSTRUMENT / COMPUTE / CONDITION / WAIT / NOTIFY / SAMPLE_SPLIT / MANUAL / UNIT_LAUNCHER
deviceType	VARCHAR	INSTRUMENT 类型必填，指定仪器类型（如 `PH_METER`）
command	VARCHAR	设备指令码（如 `MEASURE`）
executor	ENUM	COMPUTE 类型：BEAN（Spring Bean）/ LITEFLOW（规则链）
beanName / methodName	VARCHAR	BEAN 执行器：指定 Spring Bean 名和方法名
chainId	VARCHAR	LITEFLOW 执行器：规则链 ID
paramsSchema	JSON	参数 JSON Schema，供 UI 渲染表单和服务端校验
outputFields	JSON Array	步骤输出字段名列表，注册至上下文供后续步骤引用
compensateStepKey	VARCHAR	失败补偿步骤的 stepKey
runnableStandalone	BOOLEAN	是否支持单步独立调试
mockOutput	JSON	Mock 模式下返回的模拟数据

StepExecutionDO — 步骤执行记录

`pe_step_execution`

字段	类型	说明
executionId	VARCHAR	关联的流水线实例 ID

字段	类型	说明
nodeId	VARCHAR	流水线内节点 ID（来自 PipelineStepDO）
attempt	INT	重试次数，每次重试新建一行记录（1-based）
status	ENUM	PENDING/RUNNING/SUCCESS/FAILED/SKIPPED/DEAD/COMPENSATING/COMPE
waitingFor	ENUM	DEVICE_CALLBACK / MANUAL_APPROVE / EXTERNAL_EVENT / TIMER
callbackToken	VARCHAR	外部回调验证令牌（随机生成）
runMode	ENUM	NORMAL / STANDALONE / COMPENSATE / MOCK
inputSnapshot	JSON	执行时输入参数快照（含上下文解析结果）
outputData	JSON	步骤执行输出数据
waitMs / executeMs	BIGINT	排队等待耗时 / 实际执行耗时（ms）
deviceId	VARCHAR	分配执行的设备 ID

4 设备管理模块 (librax-module-device)

4.1 模块职责

设备管理模块负责实验仪器的注册、协议适配、指令执行和状态监控，采用 **设备连接器工厂模式**，将上层业务与底层通信协议解耦。

4.2 支持协议

HTTP

OkHttp 客户端，支持 WEBHOOK 推送和 POLL 轮询两种回调模式，支持 Basic/Token/HMAC 认证

TCP

原始 TCP/IP 连接，支持二进制或 ASCII 编码，适配特定厂商分析仪协议

SERIAL

串口通信（UART），可配置波特率、数据位、停止位、校验位

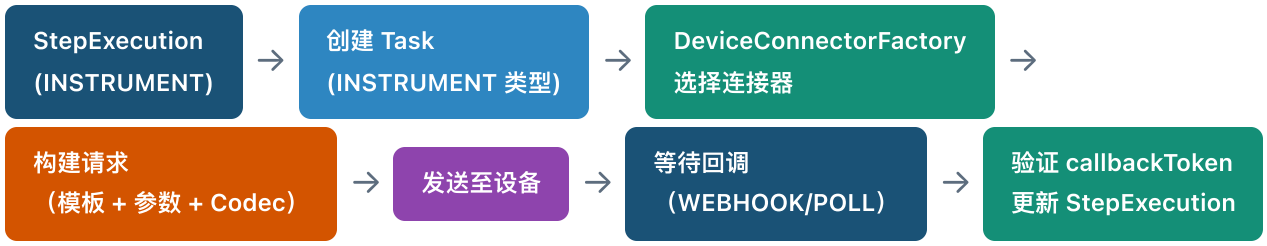
SDK

厂商 SDK 动态加载（反射），通过 sdkClass 全限定名 + sdkConfig JSON 初始化

MOCK

测试模式，返回 DeviceCommandDO.mockOutput 预设数据，不依赖物理设备

4.3 指令执行流程



4.4 设备编码/解码（Codec）

`DeviceCodecDO` 定义了请求/响应的数据转换规则，支持 JSON 字段映射、Protobuf、Avro 等格式，将设备私有协议数据转换为平台标准 JSON 格式。

4.5 设备状态监控

状态	含义	触发条件
ONLINE	设备在线，可接受指令	心跳响应正常
OFFLINE	设备离线，无法通信	心跳超时，连接中断
FAULT	设备故障，需人工介入	多次重试失败，连续错误响应

4.6 核心数据模型

DeviceInfoDO — 设备注册信息 `lab_device_info`

字段	类型	说明
deviceId	VARCHAR	设备业务 ID（如 `PH-METER-01`），全局唯一
deviceType	VARCHAR	设备类型（如 `PH_METER`），与步骤定义关联
protocol	ENUM	TCP / HTTP / SERIAL / SDK / MOCK
callbackMode	ENUM	WEBHOOK（设备主动推送）/ POLL（平台轮询查询）
authType	ENUM	NONE / BASIC / TOKEN / HMAC
authConfig	JSON	认证配置（用户名密码、Token 等）
maxConcurrent	INT	最大并发指令数，超出则排队
heartbeatIntervalMs	BIGINT	心跳检测间隔（ms），0=不检测
heartbeatCommand	VARCHAR	心跳使用的指令码
zoneCode	VARCHAR	设备所在区域，与资源管理区域对应

5 实验室管理模块 (librax-module-lab)

5.1 样本生命周期

REGISTERED 签收登记 → LOADED 上架装载 → IN_PROCESS 检测处理 → COMPLETED 检测完成 → ARCHIVED 归档保存

异常状态：REJECTED　LOST

5.2 样本衍生谱系

衍生类型说明

衍生类型	说明	父子关系
ORIGINAL	原始样本，直接采集	根节点，parentSampleId=NULL
SPLIT	分裂：一个样本拆分为多份	一父多子，子样本体积之和≤父样本
MERGE	合并：多个样本合并为一份	多父一子，SampleRelationDO 记录所有父样本
ALIQUOT	取液：取出一部分用于平行检测	父样本体积减少，子样本体积为取出量
ADD_REAGENT	加试剂：向样本中添加化学品	同一样本，记录添加物和用量

谱系追溯：通过 `rootSampleId` 可快速定位原始样本，通过 `SampleRelationDO` 表可还原完整的衍生关系树，满足 GLP/GMP 合规要求。

5.3 样本类型与容器

支持样本类型

BLOOD 血液　URINE 尿液　TISSUE 组织　WATER 水样　SOIL 土壤　AIR 气体　MIXTURE 混合物

容器类型

TUBE 试管　PLATE 微孔板　VIAL 小瓶　BAG 样品袋

5.4 材料管理

`MaterialDefDO` 定义试剂、标准品、缓冲液等实验耗材的规格信息，`MaterialInstanceDO` 跟踪实物批次、开封时间、有效期，支持危险品分级（NONE/LOW/MEDIUM/HIGH/FLAMMABLE/TOXIC）和质检规则（`MaterialCheckRuleDO`）。

6 资源调度模块 (librax-module-resource)

6.1 资源所有权模型

独占资源 (EXCLUSIVE)

- 绑定到特定区域（zoneCode），只能被该区域内的流程使用
- 例：ZONE-A 的 pH 计，不可被 ZONE-B 借用
- 无需配额限制，区域内自行管理并发
- 适合精密仪器、区域固定设备

共享资源 (SHARED)

- 跨区域共享，任意区域可申请使用
- 通过 `ZoneQuotaDO` 限制每个区域的最大借用数
- 例：AGV 小车，多个区域均可调度，但每区域最多 2 辆
- Redis 分布式锁保证原子性分配

6.2 资源分配算法

```
资源分配策略（ResourceAllocationService）：
1. 查询步骤资源需求（StepResourceReqDO）
2. 优先查找同区域内的可用独占资源
3. 独占资源不足时，检查共享资源配额
4. 使用 Redisson 分布式锁（REDLOCK）原子性执行：
   a. 检查 ZoneQuotaDO.maxBorrow - 当前占用 > 0
   b. 创建 StepResourceHoldDO 记录分配关系
   c. 更新 ResourceConfigDO.currentUsage
5. 资源不足时：任务进入等待队列，持续轮询
6. 步骤完成后：释放持有，通知等待队列重新分配
```

6.3 存储架位置管理

层级	实体	示例	说明
区域	ZoneCode（逻辑）	ZONE-A	实验室物理区域
存储单元	RackInfoDO	RACK-A（离心机架）	含坐标信息，供 AGV 导航
槽位	SlotInfoDO	RACK-A-R1-C2-L1	行-列-层精确定位
容器	SampleInfoDO.containerCode	TUBE-001	放置于槽位的样本容器

7 任务执行模块 (librax-module-task)

7.1 任务状态机

PENDING → ASSIGNED → EXECUTING → DONE

EXECUTING → FAILED → （重试次数未耗尽时自动重新 PENDING）

CANCELLED | TIMEOUT 终态，不可恢复

7.2 任务类型与执行器

任务类型	执行器类型	执行方式	回调
INSTRUMENT	DEVICE	DeviceConnector 发送仪器指令	异步（WEBHOOK/POLL）
AGV	AGV	调度 AGV 搬运（fromLocation→toLocation）	异步回调
COMPUTE	BEAN	Spring Bean 方法同步执行	同步返回
MANUAL	HUMAN	人工操作，等待人员确认完成	手动回调 API
NOTIFY	BEAN	发送通知（SMS/Email/Webhook）	同步返回

7.3 回调验证协议

```
// 外部系统回调接口
POST /flow/step-callback/{executionId}/{nodeId}

Request Body:
{
  "callbackToken": "a3f9b2c1...",   // 与 StepExecutionDO.callbackToken 比对
  "result": {
    "status": "SUCCESS",
    "output": { "value": 7.2, "unit": "pH" },
    "errorCode": null,
```

```
    "errorMsg": null
  }
}

验证逻辑:
1. 查询 StepExecutionDO (executionId + nodeId + status=RUNNING)
2. 比对 callbackToken (防重放攻击)
3. 幂等检查 (相同 token 重复回调合并处理)
4. 更新 StepExecution 状态, 触发 DAG 调度器继续
```

7.4 任务优先级与重试

优先级队列

- **CRITICAL (2)**：紧急检测，最高优先级，插队执行
- **URGENT (1)**：加急检测，优先于普通任务
- **NORMAL (0)**：常规检测，按序执行

重试配置

- **retryCount**：当前已重试次数
- **maxRetry**：最大允许重试次数
- **defaultBackoffMs**：重试退避间隔（指数递增）
- 达到 maxRetry 后标记 FAILED，触发补偿逻辑

8 系统基础模块 (librax-module-system / infra)

8.1 用户权限体系

RBAC 权限模型

- 用户（User）→ 角色（Role）→ 菜单/按钮权限（Permission）
- 角色支持部门数据权限（全部/本部门/本人）
- 接口权限通过 `@PreAuthorize` 注解声明
- 数据权限通过 SQL 拦截器动态注入过滤条件

多租户隔离

- 每个租户独立的数据集（tenant_id 字段过滤）
- 租户套餐（TenantPackage）控制功能权限边界
- TenantInterceptor 从 JWT Token 提取租户 ID
- 所有数据库查询自动追加 WHERE tenant_id = ?

8.2 基础设施服务（infra 模块）



文件服务

支持本地、S3/MinIO、阿里云 OSS、腾讯云 COS、FTP/SFTP 多种存储后端



配置中心

动态配置参数（ConfigDO），运行时修改无需重启，支持加密存储



代码生成

基于 MyBatis Plus Generator，一键生成 DO/VO/Mapper/Service/Controller



接口文档

Springdoc OpenAPI 3.0，自动生成 Swagger UI，支持在线调试

9 数据库设计

9.1 表命名规范

`pd_*`

Pipeline Definition — 流水线静态配置（定义时数据）

`pe_*`

Pipeline Execution — 流水线运行时实例（执行时数据）

`lab_*`

Laboratory — 实验室业务数据（样本、设备、材料、资源）

9.2 通用字段规范

所有数据表均继承 `BaseDO`，包含以下审计字段：

字段	类型	说明
id	BIGINT AUTO_INCREMENT	主键，雪花 ID 生成
creator	VARCHAR(64)	创建人（用户名）
create_time	DATETIME	创建时间（自动填充）
updater	VARCHAR(64)	最后更新人
update_time	DATETIME	最后更新时间（自动填充）
deleted	BIT(1)	逻辑删除标记（0=正常，1=已删）

字段	类型	说明
tenant_id	BIGINT	租户 ID（多租户数据隔离）

9.3 关键设计决策

乐观锁设计：`PipelineExecutionDO` 使用 `row_version`（整型版本号）实现乐观并发控制，状态更新时携带期望版本号：

`UPDATE pe_pipeline_execution SET status=?, row_version=row_version+1 WHERE id=? AND row_version=?`

防止高并发下状态写入冲突（如同时触发暂停和完成）。

每次重试独立记录：`StepExecutionDO.attempt` 字段标识重试次数，每次重试创建新行，保留完整的执行历史，不覆盖前次失败记录，便于问题回溯分析。

9.4 主要表结构概览

表名	中文名	记录特征	估算量级
`pd_pipeline_definition`	流水线定义	静态配置，低频写	百级
`pd_step_definition`	步骤定义	可复用组件，低频写	千级
`pd_pipeline_step`	流水线步骤关系	每条流水线 N 条节点记录	万级
`pe_pipeline_execution`	流水线执行实例	每次触发一条，高频写	百万级
`pe_step_execution`	步骤执行记录	每实例 N 步×重试次数	千万级
`pe_execution_context`	执行上下文	每实例一条，频繁更新	百万级
`pe_execution_event_log`	执行事件日志	仅追加，全量审计	亿级（需归档）
`lab_sample_info`	样本信息	含衍生样本，高频写	千万级
`lab_device_info`	设备信息	静态配置，低频写	百级
`lab_task`	任务记录	每步骤一任务，高频写	亿级（需归档）

9.5 索引设计原则

- 所有外键字段（executionId, stepKey, sampleId）建立普通索引
- 状态字段（status）+ 时间字段（createTime）建立联合索引，支持待处理任务扫描
- `callbackToken` 建立唯一索引，防止回调重复处理
- 事件日志表按月分表，历史数据定期归档至冷存储
- 执行上下文按 executionId 分区，提升并发查询效率

10 接口规范设计

10.1 URL 命名规范

HTTP Method	URL Pattern	操作
POST	`/flow/pipeline-execution/create`	创建资源
PUT	`/flow/pipeline-execution/update`	更新资源
DELETE	`/flow/pipeline-execution/delete?id=1`	单条删除
DELETE	`/flow/pipeline-execution/delete-list`	批量删除
GET	`/flow/pipeline-execution/get?id=1`	按 ID 查询
GET	`/flow/pipeline-execution/page`	分页查询
GET	`/flow/pipeline-execution/list`	列表查询（无分页）
POST	`/flow/pipeline-execution/export`	导出 Excel

10.2 统一响应格式

```
// 成功响应
{
  "code": 0,
  "msg": null,
  "data": { ... }
}

// 分页响应
{
  "code": 0,
  "data": {
    "list": [ ... ],
    "total": 1234
  }
}

// 错误响应
{
  "code": 1002001,          // 错误码（模块ID + 错误序号）
```

```
  "msg": "流水线定义不存在",
  "data": null
}
```

10.3 关键业务接口

接口	方法	说明
`/flow/pipeline-execution/create`	POST	启动一次流水线执行（支持传入 inputParams）
`/flow/pipeline-execution/pause`	PUT	暂停正在运行的流水线
`/flow/pipeline-execution/resume`	PUT	恢复暂停的流水线
`/flow/pipeline-execution/cancel`	PUT	取消流水线（终态，不可恢复）
`/flow/step-execution/standalone`	POST	单步独立执行（调试用）
`/flow/step-callback/{executionId}/{nodeId}`	POST	外部系统回调步骤完成结果
`/flow/execution-monitor/realtime`	GET	查询执行实时状态（含各步骤进度）
`/device/device-info/connect-test`	POST	测试设备连接（不执行实际指令）
`/lab/sample-info/register`	POST	样本签收登记
`/lab/sample-info/derive`	POST	样本衍生（分裂/取液/合并）
`/resource/resource-config/acquire`	POST	申请资源占用（内部调用）
`/resource/resource-config/release`	POST	释放资源占用（内部调用）

10.4 分页请求参数规范

```
GET /flow/pipeline-execution/page?pageNo=1&pageSize=20&status=RUNNING&pipelineKey=water_quality

// 所有分页接口支持的通用参数：
pageNo    : integer  // 页码（从1开始），默认1
pageSize  : integer  // 每页条数（最大100），默认10

// 响应中 total 为总记录数，list 为当前页数据
```

11 WebSocket 实时通信设计

11.1 设计目的

流水线执行过程中，前端界面需要实时展示每个步骤的执行进度、状态变化和输出结果，通过 WebSocket（STOMP 协议）实现服务端主动推送，避免频繁轮询。

11.2 推送事件类型

事件类型	触发时机	推送内容
`EXECUTION_STARTED`	流水线开始执行	executionId、pipelineKey、触发人
`STEP_QUEUED`	步骤进入就绪队列	nodeId、stepKey、预计执行时间
`STEP_STARTED`	步骤开始执行	nodeId、执行设备、inputSnapshot
`STEP_SUCCESS`	步骤执行成功	nodeId、outputData、executeMs
`STEP_FAILED`	步骤执行失败	nodeId、errorCode、errorMsg、attempt
`STEP_RETRYING`	步骤开始重试	nodeId、retryCount、backoffMs
`EXECUTION_SUCCESS`	流水线整体成功	totalMs、criticalPath
`EXECUTION_FAILED`	流水线失败	failedNodeId、errorMsg
`COMPENSATING`	开始执行补偿	compensatingNodes 列表

11.3 订阅地址规范

```
// 连接端点
ws://host:48080/ws  (STOMP over WebSocket)

// 订阅特定执行实例的事件
SUBSCRIBE /topic/execution/{executionId}

// 订阅当前用户的全局通知
SUBSCRIBE /user/queue/notify
```

```
// 心跳配置
heartbeat: [10000, 10000]  // 10s 心跳保持连接
```

12 安全架构设计

认证机制

- JWT Token 无状态认证（RS256 签名）
- AccessToken（30分钟）+ RefreshToken（7天）双 Token 机制
- Token 存储于 Redis，支持强制下线（黑名单）
- 支持社交登录（JustAuth：微信、钉钉、企业微信）
- 登录日志记录（IP、设备、成功/失败）

授权机制

- `@PreAuthorize("hasAuthority('flow:execution:create')")` 接口级鉴权
- 部门数据权限过滤（MY/DEPT/DEPT_TREE/ALL）
- 租户数据隔离（tenant_id 强制过滤）
- 敏感操作双因素验证（可选）

防护机制

- 接口限流（基于 Redis 令牌桶，@RateLimiter）
- 幂等校验（@Idempotent，Redis 去重，防重复提交）
- 回调 Token 验证（防止伪造设备回调）
- SQL 注入防护（MyBatis 参数化查询）
- XSS 防护（输入过滤 + 输出转义）

审计追踪

- 操作日志（OperateLogDO）：记录所有写操作及操作人
- 登录日志（LoginLogDO）：认证记录，含失败原因

- 执行事件日志（ExecutionEventLog）：完整流程追踪
- 任务事件（TaskEventDO）：任务生命周期记录

13 性能与可靠性设计

13.1 缓存策略

缓存对象	缓存层	TTL	更新策略
流水线定义	Caffeine（本地）	10分钟	版本变更时主动失效
步骤定义	Caffeine（本地）	10分钟	写入时主动失效
用户权限信息	Redis	30分钟	登出/角色变更时失效
租户配置	Redis	60分钟	配置更新时失效
设备状态	Redis	心跳周期×2	心跳更新
执行上下文	DB（主要）+ Redis（热数据）	执行期间	步骤完成时写 DB

13.2 数据库优化

- Dynamic Datasource：写操作走主库，读操作走从库（读写分离）
- Druid 连接池：最大连接数 100，连接有效性检测，慢 SQL 监控
- MyBatis Plus 分页插件：自动 COUNT 优化，避免全表扫描
- 高频更新表（StepExecution、Task）：乐观锁替代悲观锁，减少锁竞争
- 事件日志表：定期归档（按月分表或迁移至 ClickHouse）

13.3 并发控制

分布式锁使用场景

- 资源分配（ResourceAcquireService）：防止超额分配
- 流水线状态转换：防止并发状态写冲突
- 设备并发限制：maxConcurrent 控制并行指令数
- 任务分配：防止同一任务被多个执行器认领

可靠性保障

- Quartz 集群模式：Job 调度高可用，防止单点故障
- MQ 消息持久化：设备任务消息不丢失
- 死信队列处理：无法消费的消息人工干预
- 心跳超时检测：自动发现离线设备，告警通知
- 幂等执行：相同 callbackToken 多次回调安全合并

14 部署架构设计

14.1 容器化部署

```
# docker-compose.yml（开发环境）
services:
  librax-server:
    image: librax-server:2025.11
    ports: ["48080:48080"]
    environment:
      - SPRING_PROFILES_ACTIVE=dev
      - SPRING_DATASOURCE_URL=jdbc:mysql://mysql-local:3306/librax
      - SPRING_REDIS_HOST=redis-local
    depends_on: [mysql-local, redis-local]
    networks: [librax-network]

  mysql-local:
    image: mysql:8.0
    volumes: [./data/mysql:/var/lib/mysql]
    environment:
      - MYSQL_DATABASE=librax
    networks: [librax-network]

  redis-local:
    image: redis:7.0
    volumes: [./data/redis:/data]
    networks: [librax-network]
```

14.2 生产环境架构

负载均衡层

Nginx / SLB | SSL 终止 | WebSocket 反向代理

应用层（无状态，水平扩展）

librax-server × N 实例 | Session 共享（Redis） | 分布式定时任务（Quartz Cluster）

存储层

MySQL 主从（1主2从） | Redis Sentinel / Cluster | RocketMQ Broker 集群 | MinIO 分布式对象存储

观测层

Spring Boot Admin（服务健康） | SkyWalking（链路追踪） | Prometheus + Grafana（指标监控） | ELK（集中日志）

14.3 环境配置分离

Profile	适用场景	数据库	特点
`local`	本地开发	本机 MySQL	关闭安全校验，启用 SQL 打印
`dev`	Docker 开发	容器 MySQL	Docker Compose 网络，Flyway 自动迁移
`prod`	生产环境	RDS 主从	读写分离，关闭 Swagger，启用加密

14.4 数据库版本管理

使用 **Flyway** 管理数据库变更，版本脚本命名规范：

```
src/main/resources/db/
├─ V1.0.0__init_schema.sql        # 初始建表
├─ V1.0.1__seed_data.sql          # 基础数据
├─ V1.1.0__add_sample_module.sql  # 功能迭代
└─ V1.2.0__flow_engine_update.sql # 流程引擎更新
```

15 测试策略

单元测试

- 框架：JUnit 5 + Mockito 5
- 覆盖 Service 层业务逻辑，Mock 外部依赖
- Mapper 层使用 H2 内存数据库验证 SQL 正确性
- DAG 调度器拓扑排序、环路检测单独测试
- 状态机转换合法性验证测试

集成测试

- @SpringBootTest 启动完整上下文
- 嵌入式 Redis（Testcontainers）
- @Transactional 保证测试数据隔离回滚
- 流水线端到端执行测试（含 Mock 设备）
- 回调接口幂等性测试（重复 Token 场景）

Mock 调试模式

- DeviceInfoDO.protocol=MOCK 绕过物理设备
- DeviceCommandDO.mockOutput 预设仪器返回值
- StepDefinitionDO.mockOutput 步骤级 Mock
- 支持流水线全流程 Mock 执行，不依赖实验室环境

单步调试验证

- 通过 `/flow/step-execution/standalone` 独立触发
- 实时 WebSocket 推送执行日志

- 支持传入自定义 inputParams 测试不同场景
- 输出 outputData 和 contextData 供验证

15.1 测试资源目录规范

```
src/test/
├─ java/
│   ├─ service/       # Service 单元测试
│   ├─ dal/           # Mapper SQL 测试
│   └─ scheduler/     # DAG 调度器测试
└─ resources/
    ├─ sql/
    │   ├─ create_tables.sql   # 测试建表脚本
    │   └─ clean.sql           # 测试清理脚本
    └─ application-test.yaml   # 测试环境配置
```

> **文档维护说明**：本文档与代码同步维护，每次迭代发布前更新修订历史。设计变更须经团队评审，并在相关模块的 `CHANGELOG.md` 中记录影响范围。接口变更需同步更新 Swagger 文档注解（`@Operation` / `@Schema`）。

© 2025 Librax Team · 基于 Spring Boot 3.5 构建